Polyspace Bug Finder

Detailed Report for Project: bme690

Report Author: LibDriver

Polyspace Bug Finder: Detailed Report for Project: bme690

by Report Author: LibDriver

Published 14-Mar-2026 11:50:50

Analysis Author(s): LibDriver

Polyspace Version(s): Polyspace Bug Finder 3.2 (R2020a)

Project Version(s): 1.0

Result Folder(s):

E:\Polyspace\bme690\Module\BF_Result

Table of Contents

Chapter 1. Polyspace Bug Finder Summary 1

Chapter 2. MISRA C:2012 Guidelines 3

MISRA C:2012 Guidelines Summary - Violations by File 3

MISRA C:2012 Guidelines Violations 3

Chapter 3. Defects 86

Defects 86

Chapter 4. Appendix 1 - Configuration Settings 87

Polyspace Settings 87

Coding Standard Configuration 88

Chapter 5. Appendix 2 - Definitions 96

Chapter 1. Polyspace Bug Finder Summary

Table 1.1. Project Summary

	Count	Reviewed	Unreviewed	Pass/Fail
MISRA C:2012 Guidelines	955	955	0	Pass
Defects	0	0	0	Pass
Total	**955**	**955**	**0**	**Pass**

Table 1.2. Summary By File

File	Defects (Reviewed)	MISRA C:2012 Guidelines (Reviewed)
E:\Github\bme690\example\driver_bme690_basic.c	0 (0)	11 (11)
E:\Github\bme690\example\driver_bme690_basic.h	0 (0)	0 (0)
E:\Github\bme690\example\driver_bme690_gas.c	0 (0)	12 (12)
E:\Github\bme690\example\driver_bme690_gas.h	0 (0)	0 (0)
E:\Github\bme690\example\driver_bme690_parallel.c	0 (0)	13 (13)
E:\Github\bme690\example\driver_bme690_parallel.h	0 (0)	0 (0)
E:\Github\bme690\interface\driver_bme690_interface.h	0 (0)	0 (0)
E:\Github\bme690\interface\driver_bme690_interface_template.c	0 (0)	28 (28)
E:\Github\bme690\src\driver_bme690.c	0 (0)	491 (491)
E:\Github\bme690\src\driver_bme690.h	0 (0)	0 (0)
E:\Github\bme690\test\driver_bme690_parallel_test.c	0 (0)	54 (54)
E:\Github\bme690\test\driver_bme690_parallel_test.h	0 (0)	0 (0)
E:\Github\bme690\test\driver_bme690_read_test.c	0 (0)	40 (40)

E:\Github\bme690\test\driver_bme690_read_test.h	0 (0)	0 (0)
E:\Github\bme690\test\driver_bme690_register_test.c	0 (0)	306 (306)
E:\Github\bme690\test\driver_bme690_register_test.h	0 (0)	0 (0)

Chapter 2. MISRA C:2012 Guidelines

MISRA C:2012 Guidelines Summary - Violations by File

File	Total
E:\Github\bme690\example\driver_bme690_basic.c	11
E:\Github\bme690\example\driver_bme690_gas.c	12
E:\Github\bme690\example\driver_bme690_parallel.c	13
E:\Github\bme690\interface\driver_bme690_interface_template.c	28
E:\Github\bme690\src\driver_bme690.c	491
E:\Github\bme690\test\driver_bme690_parallel_test.c	54
E:\Github\bme690\test\driver_bme690_read_test.c	40
E:\Github\bme690\test\driver_bme690_register_test.c	306
Total	**955**

MISRA C:2012 Guidelines Violations

Table 2.1. E:\Github\bme690\example\driver_bme690_basic.c

ID	Guideline	Message	Function	Severity	Status	Comment
441	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
561	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
543	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
780	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
489	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
448	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
586	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
679	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
629	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
422	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
414	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Table 2.2. E:\Github\bme690\example\driver_bme690_gas.c

ID	Guideline	Message	Function	Severity	Status	Comment
758	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
794	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
816	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
762	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
798	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
806	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
717	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
848	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
620	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
752	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
643	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
678	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.

Table 2.3. E:\Github\bme690\example\driver_bme690_parallel.c

ID	Guideline	Message	Function	Severity	Status	Comment
453	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
716	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
426	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
451	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
565	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
616	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
473	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
456	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
418	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
443	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
461	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
610	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

508	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Table 2.4. E:\Github\bme690\interface\driver_bme690_interface_template.c

ID	Guideline	Message	Function	Severity	Status	Comment
1	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_interface_receive_callback()	Low	Not a defect	(handle == NULL)checked.
498	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
466	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
859	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_interface_receive_callback()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
437	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
524	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
459	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
487	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
421	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
530	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
440	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
464	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

2	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_interface_receive_callback()	Low	Not a defect	(handle == NULL)checked.
562	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
860	D4.14	The validity of values received from external sources shall be checked. Loop is controlled by a value from an unsecure source. Loop may be infinite.	bme690_interface_receive_callback()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
501	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
861	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_interface_receive_callback()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
479	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
785	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
703	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
491	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
744	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
446	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
485	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
480	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
3	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category.	bme690_interface_receive_callback()	Low	Not a defect	(handle == NULL)checked.

		The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)				
436	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
553	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Table 2.5. E:\Github\bme690\src\driver_bme690.c

ID	Guideline	Message	Function	Severity	Status	Comment
25	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
316	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
21	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
40	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
97	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
219	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
313	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.

33	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
35	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
336	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
44	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
48	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
52	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
192	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
46	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
204	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_change_spi_page()	Low	Not a defect	(handle == NULL)checked.
32	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the == operator has essentially unsigned type	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.

		while the right operand has essentially enum type.				
59	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the -= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
50	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
57	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
384	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
29	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
55	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
196	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_read()	Low	Not a defect	(handle == NULL)checked.
402	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the == operator has essentially unsigned type while the right operand has essentially enum type.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
58	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the -= operator has essentially unsigned type	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.

		while the right operand has essentially signed type.				
19	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
150	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
159	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
255	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
260	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
49	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
63	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
197	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
78	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.
105	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential	a_bme690_iic_spi_write()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
128	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
110	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
68	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
185	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
183	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits)	a_bme690_get_nvm_calibration()	Low	Not a defect	(handle == NULL)checked.

		which is a composite expression				
70	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	(handle == NULL)checked.
79	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
76	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	a_bme690_get_nvm_calibration()	Low	Not a defect	(handle == NULL)checked.
129	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
73	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	(handle == NULL)checked.
342	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.

404	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	a_bme690_get_nvm_calibration()	Low	Not a defect	(handle == NULL)checked.
247	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	a_bme690_compensate_temperature()	Low	Not a defect	(handle == NULL)checked.
84	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
865	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
121	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
39	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
88	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
94	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
61	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
99	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and

		Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.				drivers guarantee the safety of the operation.
81	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
102	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
72	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
66	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
213	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
866	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
71	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
206	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
867	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the

						safety of the operation.
234	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
75	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
868	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
168	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
217	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
869	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
111	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
176	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
870	D4.1	Run-time failures shall be minimized. Divisor is 0.0.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the

						safety of the operation.
96	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
82	12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
188	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
27	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
53	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
113	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
393	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
406	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
112	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the

		implementation-defined direction of rounding in some cases.				safety of the operation.
95	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
319	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the * operator has essentially floating type while the right operand has essentially signed type.	a_bme690_compensate_humidity()	Low	Not a defect	(handle == NULL)checked.
408	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the - operator has essentially floating type while the right operand has essentially signed type.	a_bme690_compensate_humidity()	Low	Not a defect	(handle == NULL)checked.
42	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
376	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
210	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
249	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
120	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
116	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and

		Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.				drivers guarantee the safety of the operation.
117	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_humidity()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
26	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the + operator has essentially signed type while the right operand has essentially floating type.	a_bme690_compensate_humidity()	Low	Not a defect	(handle == NULL)checked.
77	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the - operator has essentially signed type while the right operand has essentially floating type.	a_bme690_compensate_humidity()	Low	Not a defect	(handle == NULL)checked.
132	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category floating shall not be cast to the different essential type category unsigned.	a_bme690_compensate_heat()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
23	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the / operator has essentially signed type while the right operand has essentially floating type.	a_bme690_compensate_heat()	Low	Not a defect	(handle == NULL)checked.
131	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the + operator has essentially signed type while the right operand has essentially floating type.	a_bme690_compensate_heat()	Low	Not a defect	(handle == NULL)checked.
145	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the / operator has essentially signed type while	a_bme690_compensate_heat()	Low	Not a defect	(handle == NULL)checked.

		the right operand has essentially floating type.				
127	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the + operator has essentially signed type while the right operand has essentially floating type.	a_bme690_compensate_heat()	Low	Not a defect	(handle == NULL)checked.
123	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the - operator has essentially floating type while the right operand has essentially signed type.	a_bme690_compensate_heat()	Low	Not a defect	(handle == NULL)checked.
56	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_gas_resistance()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
98	D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood. Conversion of integer to floating-point number uses an implementation-defined direction of rounding in some cases.	a_bme690_compensate_gas_resistance()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
30	14.4	The controlling expression of an if statement and the controlling expression of an iteration-statement shall have essentially Boolean type.	a_bme690_calculate_parallel_counter()	Low	Justified	Can't be.
80	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	a_bme690_calculate_parallel_counter()	Low	Not a defect	(handle == NULL)checked.
92	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the == operator has essentially unsigned type while the right operand has essentially enum type.	bme690_init()	Low	Not a defect	(handle == NULL)checked.
138	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
229	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
252	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
85	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
195	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
89	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
389	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
203	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
399	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the == operator has essentially unsigned type while the right operand has essentially enum type.	bme690_deinit()	Low	Not a defect	(handle == NULL)checked.
838	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
420	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.

596	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
782	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
427	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
811	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
133	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
286	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
330	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
125	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
124	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
51	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.

		category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.				
115	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
136	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
135	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
269	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_spi_wire_3_data_interrupt()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
153	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_spi_wire_3_data_interrupt()	Low	Not a defect	(handle == NULL)checked.
38	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
179	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
326	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.

108	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
134	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
60	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
154	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
302	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
310	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
141	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_humidity_oversampling()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
104	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_humidity_oversampling()	Low	Not a defect	(handle == NULL)checked.
41	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
146	10.4	Both operands of an operator in which the usual arithmetic	bme690_set_temperature_oversampling()	Low	Not a defect	(handle ==

		conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.				NULL)checked.
325	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
37	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
142	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
67	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
151	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
373	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
164	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
228	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_temperature_oversampling()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.

240	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_temperature_oversampling()	Low	Not a defect	(handle == NULL)checked.
158	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
177	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
211	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
100	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
156	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
347	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
354	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
367	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
244	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.

311	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_pressure_oversampling()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
162	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_pressure_oversampling()	Low	Not a defect	(handle == NULL)checked.
20	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
216	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
262	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
165	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
163	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
171	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
208	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.

236	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
169	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_mode()	Low	Not a defect	(handle == NULL)checked.
74	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_mode()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
271	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_mode()	Low	Not a defect	(handle == NULL)checked.
45	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
91	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
331	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
22	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
174	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.

175	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
231	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
328	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
103	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the & operator has essentially enum type while the right operand has essentially signed type.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
137	10.1	Operands shall not be of an inappropriate essential type. The left operand of the & operator is of an inappropriate essential type category enum. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
337	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_filter()	Low	Not a defect	(handle == NULL)checked.
109	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_filter()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
178	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_filter()	Low	Not a defect	(handle == NULL)checked.
186	10.3	The value of an expression shall not be assigned to an object with a	bme690_set_spi_wire()	Low	Not a defect	(handle ==

		narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)				NULL)checked.
220	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
290	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
184	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
238	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
189	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
307	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
308	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
191	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_spi_wire()	Low	Not a defect	(handle == NULL)checked.
300	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category	bme690_get_spi_wire()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be

		enum.				accepted and drivers guarantee the safety of the operation.
201	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_spi_wire()	Low	Not a defect	(handle == NULL)checked.
147	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
209	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
270	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
207	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
366	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
69	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
149	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
277	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.

251	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_heater_off()	Low	Not a defect	(handle == NULL)checked.
379	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_heater_off()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
215	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_heater_off()	Low	Not a defect	(handle == NULL)checked.
122	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
292	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
317	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
218	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
199	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
87	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category enum.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
202	10.3	The value of an expression shall not be assigned to an object with a	bme690_set_run_gas()	Low	Not a defect	(handle ==

		narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)				NULL)checked.
285	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially enum type.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
226	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category enum.	bme690_set_run_gas()	Low	Not a defect	(handle == NULL)checked.
65	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_run_gas()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
374	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_run_gas()	Low	Not a defect	(handle == NULL)checked.
34	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
167	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
281	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
230	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.

227	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
233	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
289	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
321	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
160	10.1	Operands shall not be of an inappropriate essential type. The left operand of the & operator is of an inappropriate essential type category enum. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
327	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the & operator has essentially enum type while the right operand has essentially signed type.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
377	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_set_heater_profile()	Low	Not a defect	(handle == NULL)checked.
235	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_heater_profile()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
173	10.1	Operands shall not be of an inappropriate essential type.	bme690_get_heater_profile()	Low	Not a defect	(handle ==

		The right operand of the & operator is of an inappropriate essential type category signed.				NULL)checked.
352	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category floating shall not be cast to the different essential type category unsigned.	bme690_idac_heater_convert_to_register()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
322	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the * operator has essentially floating type while the right operand has essentially signed type.	bme690_idac_heater_convert_to_register()	Low	Not a defect	(handle == NULL)checked.
172	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category floating.	bme690_idac_heater_convert_to_data()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
879	D4.14	The validity of values received from external sources shall be checked. Loop is controlled by a value from an unsecure source. Loop may be infinite.	bme690_gas_wait_convert_to_register()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
360	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the + operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_gas_wait_convert_to_register()	Low	Not a defect	(handle == NULL)checked.
193	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the + operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits)	bme690_gas_wait_shared_convert_to_register()	Low	Not a defect	(handle == NULL)checked.

		which is a composite expression				
205	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_new_data_status()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
343	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_new_data_status()	Low	Not a defect	(handle == NULL)checked.
166	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_gas_measuring_status()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
239	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_gas_measuring_status()	Low	Not a defect	(handle == NULL)checked.
221	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_measuring_status()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
161	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_measuring_status()	Low	Not a defect	(handle == NULL)checked.
474	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line	File Scope	Low	Justified	Can't be.

		484).				
241	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_gas_measuring_index()	Low	Not a defect	(handle == NULL)checked.
266	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_gas_valid_status()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
386	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_gas_valid_status()	Low	Not a defect	(handle == NULL)checked.
114	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category unsigned shall not be cast to the different essential type category enum.	bme690_get_heater_stability_status()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
243	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_get_heater_stability_status()	Low	Not a defect	(handle == NULL)checked.
871	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_get_sub_measurement_index()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
872	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_get_sub_measurement_index()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
753	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line	File Scope	Low	Justified	Can't be.

		484).				
873	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_read_gas_resistance()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
254	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
157	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
200	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
407	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
170	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
242	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
119	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
237	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
361	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
383	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
294	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
329	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
222	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
101	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
278	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
246	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
214	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
880	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_gas_resistance()	Low	Justified	Can't be.
881	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 2801 to line 2805.	bme690_read_gas_resistance()	Low	Justified	Can't be.
357	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
340	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
353	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
181	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
47	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
248	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
268	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
225	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
882	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_gas_resistance()	Low	Justified	Can't be.
883	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 2855 to line 2859.	bme690_read_gas_resistance()	Low	Justified	Can't be.
250	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_gas_resistance()	Low	Not a defect	(handle == NULL)checked.
874	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source.	bme690_read_pressure()	Low	Justified	We use this function to convert driver data and drivers guarantee the

		Pointer may be NULL or may point to unknown memory.				safety of the operation.
261	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
256	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
265	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
296	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
253	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
369	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
258	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
403	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
28	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.
259	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_pressure()	Low	Not a defect	(handle == NULL)checked.

884	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_pressure()	Low	Justified	Can't be.
885	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 2977 to line 2981.	bme690_read_pressure()	Low	Justified	Can't be.
886	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_pressure()	Low	Justified	Can't be.
887	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 2988 to line 2992.	bme690_read_pressure()	Low	Justified	Can't be.
888	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_pressure()	Low	Justified	Can't be.
889	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3021 to line 3025.	bme690_read_pressure()	Low	Justified	Can't be.
890	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_pressure()	Low	Justified	Can't be.
891	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3032 to line 3036.	bme690_read_pressure()	Low	Justified	Can't be.
875	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_read_temperature()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
212	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
43	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
323	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.

368	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
54	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
263	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
139	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
232	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
356	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
106	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_temperature()	Low	Not a defect	(handle == NULL)checked.
892	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature()	Low	Justified	Can't be.
893	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3145 to line 3149.	bme690_read_temperature()	Low	Justified	Can't be.
894	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature()	Low	Justified	Can't be.
895	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3178 to line 3182.	bme690_read_temperature()	Low	Justified	Can't be.
876	D4.14	The validity of values received from external sources shall be	bme690_read_humidity()	Low	Justified	We use this function to

		checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.				convert driver data and drivers guarantee the safety of the operation.
273	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
126	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
299	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
305	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
335	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
194	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
267	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
382	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
264	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
334	10.1	Operands shall not be of an inappropriate essential type.	bme690_read_humidity()	Low	Not a defect	(handle ==

		The right operand of the & operator is of an inappropriate essential type category signed.				NULL)checked.
896	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_humidity()	Low	Justified	Can't be.
897	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3293 to line 3297.	bme690_read_humidity()	Low	Justified	Can't be.
348	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
898	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_humidity()	Low	Justified	Can't be.
899	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3302 to line 3306.	bme690_read_humidity()	Low	Justified	Can't be.
900	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_humidity()	Low	Justified	Can't be.
901	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3335 to line 3339.	bme690_read_humidity()	Low	Justified	Can't be.
272	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read_humidity()	Low	Not a defect	(handle == NULL)checked.
902	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_humidity()	Low	Justified	Can't be.
903	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3344 to line 3348.	bme690_read_humidity()	Low	Justified	Can't be.
288	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
279	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.

		The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)				
301	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
303	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
284	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
223	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
280	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
338	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
339	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
282	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
904	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
905	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3464 to line 3468.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.

906	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
907	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3475 to line 3479.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
295	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
908	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
909	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3484 to line 3488.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
910	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
911	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3517 to line 3521.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
912	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
913	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3528 to line 3532.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
187	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read_temperature_pressure_humidity()	Low	Not a defect	(handle == NULL)checked.
914	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
915	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3537 to line 3541.	bme690_read_temperature_pressure_humidity()	Low	Justified	Can't be.
412	5.8	Identifiers that define objects or functions with external linkage shall be unique.	File Scope	Low	Justified	Can't be.

		variable index conflicts with the function name index (string.h line 484).				
315	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
90	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
148	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
390	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_read()	Low	Not a defect	(handle == NULL)checked.
291	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
24	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
130	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
257	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
375	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
293	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential	bme690_read()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
916	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
917	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3672 to line 3676.	bme690_read()	Low	Justified	Can't be.
918	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
919	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3683 to line 3687.	bme690_read()	Low	Justified	Can't be.
297	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read()	Low	Not a defect	(handle == NULL)checked.
920	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
921	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3692 to line 3696.	bme690_read()	Low	Justified	Can't be.
298	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
358	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
350	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
144	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
371	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type.	bme690_read()	Low	Not a defect	(handle == NULL)checked.

		The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression				
398	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
287	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
922	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
923	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3721 to line 3725.	bme690_read()	Low	Justified	Can't be.
198	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
924	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
925	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3767 to line 3771.	bme690_read()	Low	Justified	Can't be.
926	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
927	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3778 to line 3782.	bme690_read()	Low	Justified	Can't be.
36	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_read()	Low	Not a defect	(handle == NULL)checked.
928	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
929	2.1	A project shall not contain unreachable code. If-condition always evaluates to false.	bme690_read()	Low	Justified	Can't be.

		Dead branch from line 3787 to line 3791.				
93	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
83	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
309	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
306	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
312	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_read()	Low	Not a defect	(handle == NULL)checked.
140	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
314	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
930	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_read()	Low	Justified	Can't be.
931	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3816 to line 3820.	bme690_read()	Low	Justified	Can't be.
118	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_read()	Low	Not a defect	(handle == NULL)checked.
391	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
107	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
324	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
932	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
933	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3944 to line 3948.	bme690_parallel_progress()	Low	Justified	Can't be.
934	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
935	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3955 to line 3959.	bme690_parallel_progress()	Low	Justified	Can't be.
318	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
936	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
937	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3964 to line 3968.	bme690_parallel_progress()	Low	Justified	Can't be.
224	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
394	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

274	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
938	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
939	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 3981 to line 3985.	bme690_parallel_progress()	Low	Justified	Can't be.
190	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type uint8_t shall not be cast to the wider essential type uint16_t.	bme690_parallel_progress()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
332	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
320	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
362	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
333	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
180	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
341	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

359	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
86	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
64	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
365	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
940	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
941	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4072 to line 4076.	bme690_parallel_progress()	Low	Justified	Can't be.
942	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
943	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4083 to line 4087.	bme690_parallel_progress()	Low	Justified	Can't be.
344	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
944	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
945	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4092 to line 4096.	bme690_parallel_progress()	Low	Justified	Can't be.
405	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

		The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression				
349	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
397	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
946	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
947	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4109 to line 4113.	bme690_parallel_progress()	Low	Justified	Can't be.
387	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type uint8_t shall not be cast to the wider essential type uint16_t.	bme690_parallel_progress()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
351	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
346	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
380	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
355	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
276	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

		category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.				
364	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
370	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
363	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
385	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
345	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
948	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
949	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4200 to line 4204.	bme690_parallel_progress()	Low	Justified	Can't be.
950	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
951	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4211 to line 4215.	bme690_parallel_progress()	Low	Justified	Can't be.
283	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type unsigned on 32 bits) is assigned to an object with a narrower essential type (unsigned on 16 bits)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
952	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.

953	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4220 to line 4224.	bme690_parallel_progress()	Low	Justified	Can't be.
388	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the | operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
372	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
378	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
954	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_progress()	Low	Justified	Can't be.
955	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 4237 to line 4241.	bme690_parallel_progress()	Low	Justified	Can't be.
381	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type uint8_t shall not be cast to the wider essential type uint16_t.	bme690_parallel_progress()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
304	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
182	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
143	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.

		type category signed.				
31	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
62	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
155	10.1	Operands shall not be of an inappropriate essential type. The right operand of the |= operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
395	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the |= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
401	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_progress()	Low	Not a defect	(handle == NULL)checked.
245	10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type. The left operand of the + operator shall not have wider essential type (unsigned on 16 bits) than the right operand (unsigned on 8 bits) which is a composite expression	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
152	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the &= operator has essentially unsigned type while the right operand has essentially signed type.	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
275	10.1	Operands shall not be of an inappropriate essential type. The right operand of the &= operator is of an inappropriate essential type category signed.	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
400	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.

		object with a different essential type category (unsigned)				
392	10.1	Operands shall not be of an inappropriate essential type. The operand of the ~ operator is of an inappropriate essential type category signed.	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
396	10.1	Operands shall not be of an inappropriate essential type. The left operand of the << operator is of an inappropriate essential type category signed.	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
409	10.1	Operands shall not be of an inappropriate essential type. The right operand of the & operator is of an inappropriate essential type category signed.	bme690_parallel_config()	Low	Not a defect	(handle == NULL)checked.
877	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_set_reg()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
878	D4.14	The validity of values received from external sources shall be checked. Dereferenced pointer is from an unsecure source. Pointer may be NULL or may point to unknown memory.	bme690_get_reg()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.

Table 2.6. E:\Github\bme690\test\driver_bme690_parallel_test.c

ID	Guideline	Message	Function	Severity	Status	Comment
4	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	a_receive_callback()	Low	Not a defect	(handle == NULL)checked.
521	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
515	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
488	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
601	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
818	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

528	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
531	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
529	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
560	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
514	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
5	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	a_receive_callback()	Low	Not a defect	(handle == NULL)checked.
518	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
523	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
787	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
507	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
691	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
432	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
747	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
783	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
522	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
463	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
6	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category enum) is assigned to an object with a different essential type category (unsigned)	a_receive_callback()	Low	Not a defect	(handle == NULL)checked.
683	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
525	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
539	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
537	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
533	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
738	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
470	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
810	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
544	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
419	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
746	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
517	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
635	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
546	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

636	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
750	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
766	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
457	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
559	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
778	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
460	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
510	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
492	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
468	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
516	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
509	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
513	2.2	There shall be no dead code. The call to function bme690_interface_delay_ms has no effect.	File Scope	Low	Justified	delay function.
862	14.3	Controlling expressions shall not be invariant. If condition is always false.	bme690_parallel_test()	Low	Justified	Can't be.
863	2.1	A project shall not contain unreachable code. If-condition always evaluates to false. Dead branch from line 324 to line 351.	bme690_parallel_test()	Low	Justified	Can't be.
772	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

506	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Table 2.7. E:\Github\bme690\test\driver_bme690_read_test.c

ID	Guideline	Message	Function	Severity	Status	Comment
433	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
855	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
851	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
826	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
434	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
809	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
701	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
853	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
626	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
849	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
840	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
415	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
828	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
770	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
834	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
595	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
815	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
796	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
790	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
827	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
858	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
431	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
813	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
571	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
788	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
755	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
791	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
682	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
642	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
784	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

417	2.2	There shall be no dead code. The call to function bme690_interface_delay_ms has no effect.	File Scope	Low	Justified	delay function.
864	D4.14	The validity of values received from external sources shall be checked. Loop is controlled by a value from an unsecure source. Loop may be infinite.	bme690_read_test()	Low	Justified	We use this function to convert driver data and drivers guarantee the safety of the operation.
776	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
774	2.2	There shall be no dead code. The call to function bme690_interface_delay_ms has no effect.	File Scope	Low	Justified	delay function.
493	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
763	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
760	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
697	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
575	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
779	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Table 2.8. E:\Github\bme690\test\driver_bme690_register_test.c

ID	Guideline	Message	Function	Severity	Status	Comment
411	5.8	Identifiers that define objects or functions with external linkage shall be unique. variable index conflicts with the function name index (string.h line 484).	File Scope	Low	Justified	Can't be.
751	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
496	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
607	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
748	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
654	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
594	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
481	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
545	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
857	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
843	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
618	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
799	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
439	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
502	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
740	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
757	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
442	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
563	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
718	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

727	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
698	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
839	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
549	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
733	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
768	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
844	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
597	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
445	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
856	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
692	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
773	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
690	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
490	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
824	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
706	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
700	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
723	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
655	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
617	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
759	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
696	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
627	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
793	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
789	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
712	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
686	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
754	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
710	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
566	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
589	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
576	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
732	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

656	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
673	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
814	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
832	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
633	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
817	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
714	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
713	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
771	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
425	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
476	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
469	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
430	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
663	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
555	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
658	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
685	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
702	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
584	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
645	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
651	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
580	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
639	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
695	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
719	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
822	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
795	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
676	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
767	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
687	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
495	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
721	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
825	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

637	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
821	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
632	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
526	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
850	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
689	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
648	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
505	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
745	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
458	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
435	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
628	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
705	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
624	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
579	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
641	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
833	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
455	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
660	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
621	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
583	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
413	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
482	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
619	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
735	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
615	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
614	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
551	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
802	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
804	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
684	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
845	2.2	There shall be no dead code. The call to function bme690_interface_delay_ms has no effect.	File Scope	Low	Justified	delay function.
800	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

609	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
608	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
605	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
462	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
708	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
504	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
665	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
613	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
667	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
677	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
599	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
742	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
582	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
661	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
630	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
438	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
536	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
540	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
416	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
657	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
743	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
730	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
699	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
640	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
593	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
478	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
592	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
842	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
836	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
598	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
803	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
829	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
726	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

532	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
503	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
606	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
734	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
500	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
644	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
786	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
590	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
424	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
797	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
653	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
588	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
452	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
585	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
449	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
715	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
472	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
467	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
837	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
650	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
556	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
777	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
631	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
477	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
568	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
729	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
581	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
841	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
497	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
761	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
739	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
494	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
672	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

749	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
483	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
484	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
649	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
664	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
577	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
846	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
602	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
572	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
852	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
741	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
622	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
781	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
722	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
564	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
428	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
808	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
668	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
512	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
623	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
835	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
587	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
694	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
819	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
534	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
535	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
578	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
558	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
638	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
659	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
567	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
670	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
429	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

454	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
736	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
573	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
557	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
688	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
801	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
552	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
830	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
520	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
831	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
737	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
725	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
671	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
554	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
675	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
775	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
720	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
646	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
769	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
423	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
820	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
709	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
450	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
711	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
674	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
764	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
662	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
765	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
756	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
410	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
707	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
499	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
591	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

550	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
486	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
570	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
693	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
519	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
16	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
652	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
625	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
475	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
444	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
548	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
8	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
7	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the % operator has essentially signed type while the right operand has essentially unsigned type.	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
669	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

854	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
792	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
569	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
465	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
11	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
9	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the % operator has essentially signed type while the right operand has essentially unsigned type.	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
731	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
681	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
604	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
807	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
603	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
13	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
12	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the % operator has essentially signed type while the right operand has essentially unsigned type.	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.

647	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
728	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
612	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
847	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
805	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
18	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category signed shall not be cast to the different essential type category floating.	bme690_register_test()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
14	10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category. The left operand of the / operator has essentially floating type while the right operand has essentially signed type.	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
611	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
724	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
547	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
17	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
680	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
471	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

704	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
15	10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type. The value of the composite expression of essential type category signed shall not be cast to the different essential type category floating.	bme690_register_test()	Low	Not a defect	We use enumeration to define driver configuration, which is a friendly programming method and should be accepted and drivers guarantee the safety of the operation.
511	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
542	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
634	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
823	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
447	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
812	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
10	10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category. The expression (of essential type category signed) is assigned to an object with a different essential type category (unsigned)	bme690_register_test()	Low	Not a defect	(handle == NULL)checked.
666	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
538	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
574	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
541	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.
600	2.2	There shall be no dead code.	File Scope	Low	Justified	print function.

		The call to function bme690_interface_debug_print has no effect.				
527	2.2	There shall be no dead code. The call to function bme690_interface_debug_print has no effect.	File Scope	Low	Justified	print function.

Chapter 3. Defects

Defects

No defects were found.

Chapter 4. Appendix 1 - Configuration Settings

Polyspace Settings

Option	Value
-author	LibDriver
-bug-finder	true
-checkers	ALIGNMENT_CHANGE, ASSERT, ATOMIC_VAR_ACCESS_TWICE, ATOMIC_VAR_SEQUENCE_NOT_ATOMIC, BAD_EQUAL_EQUAL_USE, BAD_EQUAL_USE, BAD_FREE, BAD_LOCK, BAD_PTR_SCALING, BAD_UNLOCK, CHARACTER_MISUSE, CHAR_EOF_CONFUSED, CLOSED_RESOURCE_USE, CONSTANT_OBJECT_WRITE, DATA_RACE, DATA_RACE_STD_LIB, DEADLOCK, DECL_MISMATCH, DOUBLE_DEALLOCATION, DOUBLE_LOCK, DOUBLE_RESOURCE_CLOSE, DOUBLE_RESOURCE_OPEN, DOUBLE_UNLOCK, ERRNO_MISUSE, FILE_OBJECT_MISUSE, FLEXIBLE_ARRAY_MEMBER_STRUCT_MISUSE, FLOAT_ABSORPTION, FLOAT_CONV_OVFL, FLOAT_STD_LIB, FREED_PTR, FUNC_CAST, IMPROPER_ARRAY_INIT, INLINE_CONSTRAINT_NOT_RESPECTED, INT_CONV_OVFL, INT_STD_LIB, INT_ZERO_DIV, INVALID_ENV_POINTER, INVALID_MEMORY_ASSUMPTION, INVALID_VA_LIST_ARG, IO_INTERLEAVING, LOCAL_ADDR_ESCAPE, MACRO_USED_AS_OBJECT, MEMCMP_PADDING_DATA, MEMCMP_STRINGS, MEM_STD_LIB, MISSING_ERRNO_RESET, MISSING_NULL_CHAR, MISSING_RETURN, NON_INIT_PTR, NON_INIT_VAR, NON_POSITIVE_VLA_SIZE, NULL_PTR, OPERATOR_PRECEDENCE, OTHER_STD_LIB, OUT_BOUND_ARRAY, OUT_BOUND_PTR, PARTIALLY_ACCESSED_ARRAY, PRE_DIRECTIVE_MACRO_ARG, PRE_UCNAME_JOIN_TOKENS, PTR_CAST, PTR_SIZEOF_MISMATCH, PTR_TO_DIFF_ARRAY, PUTENV_AUTO_VAR, READ_ONLY_RESOURCE_WRITE, RESOURCE_LEAK, SIDE_EFFECT_IGNORED, SIGN_CHANGE, SIG_HANDLER_CALLING_SIGNAL, SIG_HANDLER_COMP_EXCP_RETURN, SIG_HANDLER_ERRNO_MISUSE, SIG_HANDLER_SHARED_OBJECT, SIZEOF_MISUSE, STD_FUNC_ARG_MISMATCH, STREAM_WITH_SIDE_EFFECT, STRING_FORMAT, STRLIB_BUFFER_OVERFLOW, STRLIB_BUFFER_UNDERFLOW, STR_FORMAT_BUFFER_OVERFLOW, STR_STD_LIB, TEMP_OBJECT_ACCESS, TOO_MANY_VA_ARG_CALLS, TYPEDEF_MISMATCH, UINT_CONV_OVFL, UNPROTOTYPED_FUNC_CALL, UNREACHABLE, USELESS_IF, USELESS_WRITE, VAR_SHADOWING, VA_ARG_INCORRECT_TYPE, VA_START_INCORRECT_TYPE, VA_START_MISUSE
-compiler	iar
-D	__TID__=14,__SIZE_T_TYPE__=unsigned int,__PTRDIFF_T_TYPE__=signed int,__IAR_SYSTEMS_ICC=1
-date	14/03/2026
-dos	true
-I	E:\Github\bme690\src,E:\Github\bme690\example,E:\Github\bme690\interface,E:\Github\bme690\test
-import-comments	E:\Polyspace\bme690\Module\BF_Result\comments_bak
-lang	C

-little-endian	true
-logical-signed-right-shift	true
-misra3	mandatory-required
-prog	bme690
-results-dir	E:\Polyspace\bme690\Module\BF_Result
-sfr-types	sfr8=8,sfr16=16,sfr32=32,sfr=8
-target	mcpu
-verif-version	1.0

Coding Standard Configuration

Table 4.1. MISRA C:2012 Guidelines Configuration

Guideline	Description	Mode	Comment	Enabled
D1.1	Any implementation-defined behaviour on which the output of the program depends shall be documented and understood.	required	-	yes
D2.1	All source files shall compile without any compilation errors.	required	-	yes
D3.1	All code shall be traceable to documented requirements.	required	Not enforceable	no
D4.1	Run-time failures shall be minimized.	required	-	yes
D4.2	All usage of assembly language should be documented.	advisory	Not enforceable	no
D4.3	Assembly language shall be encapsulated and isolated.	required	-	yes
D4.4	Sections of code should not be "commented out".	advisory	Not implemented	no
D4.5	Identifiers in the same name space with overlapping visibility should be typographically unambiguous.	advisory	-	no
D4.6	typedefs that indicate size and signedness should be used in place of the basic numerical types.	advisory	-	no
D4.7	If a function returns error information, then that error information shall be tested.	required	-	yes
D4.8	If a pointer to a structure or union is never dereferenced within a translation unit, then the implementation of the object should be hidden.	advisory	-	no
D4.9	A function should be used in preference to a function-like macro where they are interchangeable.	advisory	-	no
D4.10	Precautions shall be taken in order to prevent the contents of a header file being included more than once.	required	-	yes

D4.11	The validity of values passed to library functions shall be checked.	required	-	yes
D4.12	Dynamic memory allocation shall not be used.	required	-	yes
D4.13	Functions which are designed to provide operations on a resource should be called in an appropriate sequence.	advisory	-	no
D4.14	The validity of values received from external sources shall be checked.	required	-	yes
1.1	The program shall contain no violations of the standard C syntax and constraints, and shall not exceed the implementation's translation limits.	required	-	yes
1.2	Language extensions should not be used.	advisory	-	no
1.3	There shall be no occurrence of undefined or critical unspecified behaviour.	required	-	yes
2.1	A project shall not contain unreachable code.	required	-	yes
2.2	There shall be no dead code.	required	-	yes
2.3	A project should not contain unused type declarations.	advisory	-	no
2.4	A project should not contain unused tag declarations.	advisory	-	no
2.5	A project should not contain unused macro declarations.	advisory	-	no
2.6	A function should not contain unused label declarations.	advisory	-	no
2.7	There should be no unused parameters in functions.	advisory	-	no
3.1	The character sequences /* and // shall not be used within a comment.	required	-	yes
3.2	Line-splicing shall not be used in // comments.	required	-	yes
4.1	Octal and hexadecimal escape sequences shall be terminated.	required	-	yes
4.2	Trigraphs should not be used.	advisory	-	no
5.1	External identifiers shall be distinct.	required	-	yes
5.2	Identifiers declared in the same scope and name space shall be distinct.	required	-	yes
5.3	An identifier declared in an inner scope shall not hide an identifier declared in an outer scope.	required	-	yes
5.4	Macro identifiers shall be distinct.	required	-	yes
5.5	Identifiers shall be distinct from macro names.	required	-	yes
5.6	A typedef name shall be a unique identifier.	required	-	yes
5.7	A tag name shall be a unique identifier.	required	-	yes
5.8	Identifiers that define objects or functions with external linkage shall be unique.	required	-	yes

5.9	Identifiers that define objects or functions with internal linkage should be unique.	advisory	-	no
6.1	Bit-fields shall only be declared with an appropriate type.	required	-	yes
6.2	Single-bit named bit fields shall not be of a signed type.	required	-	yes
7.1	Octal constants shall not be used.	required	-	yes
7.2	A "u" or "U" suffix shall be applied to all integer constants that are represented in an unsigned type.	required	-	yes
7.3	The lowercase character "l" shall not be used in a literal suffix.	required	-	yes
7.4	A string literal shall not be assigned to an object unless the object's type is "pointer to const-qualified char".	required	-	yes
8.1	Types shall be explicitly specified.	required	-	yes
8.2	Function types shall be in prototype form with named parameters.	required	-	yes
8.3	All declarations of an object or function shall use the same names and type qualifiers.	required	-	yes
8.4	A compatible declaration shall be visible when an object or function with external linkage is defined.	required	-	yes
8.5	An external object or function shall be declared once in one and only one file.	required	-	yes
8.6	An identifier with external linkage shall have exactly one external definition.	required	-	yes
8.7	Functions and objects should not be defined with external linkage if they are referenced in only one translation unit.	advisory	-	no
8.8	The static storage class specifier shall be used in all declarations of objects and functions that have internal linkage.	required	-	yes
8.9	An object should be defined at block scope if its identifier only appears in a single function.	advisory	-	no
8.10	An inline function shall be declared with the static storage class.	required	-	yes
8.11	When an array with external linkage is declared, its size should be explicitly specified.	advisory	-	no
8.12	Within an enumerator list, the value of an implicitly-specified enumeration constant shall be unique.	required	-	yes
8.13	A pointer should point to a const-qualified type whenever possible.	advisory	-	no
8.14	The restrict type qualifier shall not be used.	required	-	yes
9.1	The value of an object with automatic storage duration shall not be read before it has been set.	mandatory	-	yes
9.2	The initializer for an aggregate or union shall be enclosed in braces.	required	-	yes
9.3	Arrays shall not be partially initialized.	required	-	yes
9.4	An element of an object shall not be initialized more than once.	required	-	yes

9.5	Where designated initializers are used to initialize an array object the size of the array shall be specified explicitly.	required	-	yes
10.1	Operands shall not be of an inappropriate essential type.	required	-	yes
10.2	Expressions of essentially character type shall not be used inappropriately in addition and subtraction operations.	required	-	yes
10.3	The value of an expression shall not be assigned to an object with a narrower essential type or of a different essential type category.	required	-	yes
10.4	Both operands of an operator in which the usual arithmetic conversions are performed shall have the same essential type category.	required	-	yes
10.5	The value of an expression should not be cast to an inappropriate essential type.	advisory	-	no
10.6	The value of a composite expression shall not be assigned to an object with wider essential type.	required	-	yes
10.7	If a composite expression is used as one operand of an operator in which the usual arithmetic conversions are performed then the other operand shall not have wider essential type.	required	-	yes
10.8	The value of a composite expression shall not be cast to a different essential type category or a wider essential type.	required	-	yes
11.1	Conversions shall not be performed between a pointer to a function and any other type.	required	-	yes
11.2	Conversions shall not be performed between a pointer to an incomplete type and any other type.	required	-	yes
11.3	A cast shall not be performed between a pointer to object type and a pointer to a different object type.	required	-	yes
11.4	A conversion should not be performed between a pointer to object and an integer type.	advisory	-	no
11.5	A conversion should not be performed from pointer to void into pointer to object.	advisory	-	no
11.6	A cast shall not be performed between pointer to void and an arithmetic type.	required	-	yes
11.7	A cast shall not be performed between pointer to object and a non-integer arithmetic type.	required	-	yes
11.8	A cast shall not remove any const or volatile qualification from the type pointed to by a pointer.	required	-	yes
11.9	The macro NULL shall be the only permitted form of integer null pointer constant.	required	-	yes
12.1	The precedence of operators within expressions should be made explicit.	advisory	-	no
12.2	The right hand operand of a shift operator shall lie in the range zero to one less than the width in bits of the essential type of the left hand operand.	required	-	yes
12.3	The comma operator should not be used	advisory	-	no
12.4	Evaluation of constant expressions should not lead to unsigned integer wrap-around.	advisory	-	no
12.5	The sizeof operator shall not have an operand which is a function parameter declared as "array of	mandatory	-	yes

	type".			
13.1	Initializer lists shall not contain persistent side effects.	required	-	yes
13.2	The value of an expression and its persistent side effects shall be the same under all permitted evaluation orders.	required	-	yes
13.3	A full expression containing an increment (++) or decrement (--) operator should have no other potential side effects other than that caused by the increment or decrement operator.	advisory	-	no
13.4	The result of an assignment operator should not be used.	advisory	-	no
13.5	The right hand operand of a logical && or || operator shall not contain persistent side effects.	required	-	yes
13.6	The operand of the sizeof operator shall not contain any expression which has potential side effects.	mandatory	-	yes
14.1	A loop counter shall not have essentially floating type.	required	-	yes
14.2	A for loop shall be well-formed.	required	-	yes
14.3	Controlling expressions shall not be invariant.	required	-	yes
14.4	The controlling expression of an if statement and the controlling expression of an iteration-statement shall have essentially Boolean type.	required	-	yes
15.1	The goto statement should not be used.	advisory	-	no
15.2	The goto statement shall jump to a label declared later in the same function.	required	-	yes
15.3	Any label referenced by a goto statement shall be declared in the same block, or in any block enclosing the goto statement.	required	-	yes
15.4	There should be no more than one break or goto statement used to terminate any iteration statement.	advisory	-	no
15.5	A function should have a single point of exit at the end.	advisory	-	no
15.6	The body of an iteration-statement or a selection-statement shall be a compound-statement.	required	-	yes
15.7	All if ... else if constructs shall be terminated with an else statement.	required	-	yes
16.1	All switch statements shall be well-formed.	required	-	yes
16.2	A switch label shall only be used when the most closely-enclosing compound statement is the body of a switch statement.	required	-	yes
16.3	An unconditional break statement shall terminate every switch-clause.	required	-	yes
16.4	Every switch statement shall have a default label.	required	-	yes
16.5	A default label shall appear as either the first or the last switch label of a switch statement.	required	-	yes
16.6	Every switch statement shall have at least two switch-clauses.	required	-	yes

16.7	A switch-expression shall not have essentially Boolean type.	required	-	yes
17.1	The features of <stdarg.h> shall not be used.	required	-	yes
17.2	Functions shall not call themselves, either directly or indirectly.	required	-	yes
17.3	A function shall not be declared implicitly.	mandatory	-	yes
17.4	All exit paths from a function with non-void return type shall have an explicit return statement with an expression.	mandatory	-	yes
17.5	The function argument corresponding to a parameter declared to have an array type shall have an appropriate number of elements.	advisory	-	no
17.6	The declaration of an array parameter shall not contain the static keyword between the [ ].	mandatory	-	yes
17.7	The value returned by a function having non-void return type shall be used.	required	-	yes
17.8	A function parameter should not be modified.	advisory	-	no
18.1	A pointer resulting from arithmetic on a pointer operand shall address an element of the same array as that pointer operand.	required	-	yes
18.2	Subtraction between pointers shall only be applied to pointers that address elements of the same array.	required	-	yes
18.3	The relational operators >, >=, < and <= shall not be applied to objects of pointer type except where they point into the same object.	required	-	yes
18.4	The +, -, += and -= operators should not be applied to an expression of pointer type.	advisory	-	no
18.5	Declarations should contain no more than two levels of pointer nesting.	advisory	-	no
18.6	The address of an object with automatic storage shall not be copied to another object that persists after the first object has ceased to exist.	required	-	yes
18.7	Flexible array members shall not be declared.	required	-	yes
18.8	Variable-length array types shall not be used.	required	-	yes
19.1	An object shall not be assigned or copied to an overlapping object.	mandatory	-	yes
19.2	The union keyword should not be used.	advisory	-	no
20.1	#include directives should only be preceded by preprocessor directives or comments.	advisory	-	no
20.2	The ', " or \ characters and the /* or // character sequences shall not occur in a header file name.	required	-	yes
20.3	The #include directive shall be followed by either a <filename> or "filename"sequence.	required	-	yes
20.4	A macro shall not be defined with the same name as a keyword.	required	-	yes

20.5	#undef should not be used.	advisory	-	no
20.6	Tokens that look like a preprocessing directive shall not occur within a macro argument.	required	-	yes
20.7	Expressions resulting from the expansion of macro parameters shall be enclosed in parentheses.	required	-	yes
20.8	The controlling expression of a #if or #elif preprocessing directive shall evaluate to 0 or 1.	required	-	yes
20.9	All identifiers used in the controlling expression of #if or #elif preprocessing directives shall be #define'd before evaluation.	required	-	yes
20.10	The # and ## preprocessor operators should not be used.	advisory	-	no
20.11	A macro parameter immediately following a # operator shall not immediately be followed by a ## operator.	required	-	yes
20.12	A macro parameter used as an operand to the # or ## operators, which is itself subject to further macro replacement, shall only be used as an operand to these operators.	required	-	yes
20.13	A line whose first token is # shall be a valid preprocessing directive.	required	-	yes
20.14	All #else, #elif and #endif preprocessor directives shall reside in the same file as the #if, #ifdef or #ifndef directive to which they are related.	required	-	yes
21.1	#define and #undef shall not be used on a reserved identifier or reserved macro name.	required	-	yes
21.2	A reserved identifier or macro name shall not be declared.	required	-	yes
21.3	The memory allocation and deallocation functions of <stdlib.h> shall not be used.	required	-	yes
21.4	The standard header file <setjmp.h> shall not be used.	required	-	yes
21.5	The standard header file <signal.h> shall not be used.	required	-	yes
21.6	The Standard Library input/output functions shall not be used.	required	-	yes
21.7	The atof, atoi, atol, and atoll functions of <stdlib.h> shall not be used.	required	-	yes
21.8	The library functions abort, exit and system of <stdlib.h> shall not be used.	required	-	yes
21.9	The library functions bsearch and qsort of <stdlib.h> shall not be used.	required	-	yes
21.10	The Standard Library time and date functions shall not be used.	required	-	yes
21.11	The standard header file <tgmath.h> shall not be used.	required	-	yes
21.12	The exception handling features of <fenv.h> should not be used.	advisory	-	no
21.13	Any value passed to a function in <ctype.h> shall be representable as an unsigned char or be the value EOF.	mandatory	-	yes
21.14	The Standard Library function memcmp shall not be used to compare null terminated strings.	required	-	yes

21.15	The pointer arguments to the Standard Library functions memcpy, memmove and memcmp shall be pointers to qualified or unqualified versions of compatible types.	required	-	yes
21.16	The pointer arguments to the Standard Library function memcmp shall point to either a pointer type, an essentially signed type, an essentially unsigned type, an essentially Boolean type or an essentially enum type.	required	-	yes
21.17	Use of the string handling functions from <string.h> shall not result in accesses beyond the bounds of the objects referenced by their pointer parameters.	mandatory	-	yes
21.18	The size_t argument passed to any function in <string.h> shall have an appropriate value.	mandatory	-	yes
21.19	The pointers returned by the Standard Library functions localeconv, getenv, setlocale or, strerror shall only be used as if they have pointer to const-qualified type.	mandatory	-	yes
21.20	The pointer returned by the Standard Library functions asctime, ctime, gmtime, localtime, localeconv, getenv, setlocale or strerror shall not be used following a subsequent call to the same function.	mandatory	-	yes
22.1	All resources obtained dynamically by means of Standard Library functions shall be explicitly released.	required	-	yes
22.2	A block of memory shall only be freed if it was allocated by means of a Standard Library function.	mandatory	-	yes
22.3	The same file shall not be open for read and write access at the same time on different streams.	required	-	yes
22.4	There shall be no attempt to write to a stream which has been opened as read-only.	mandatory	-	yes
22.5	A pointer to a FILE object shall not be dereferenced.	mandatory	-	yes
22.6	The value of a pointer to a FILE shall not be used after the associated stream has been closed.	mandatory	-	yes
22.7	The macro EOF shall only be compared with the unmodified return value from any Standard Library function capable of returning EOF.	required	-	yes
22.8	The value of errno shall be set to zero prior to a call to an errno-setting-function.	required	-	yes
22.9	The value of errno shall be tested against zero after calling an errno-setting-function.	required	-	yes
22.10	The value of errno shall only be tested when the last function to be called was an errno-setting-function.	required	-	yes

Chapter 5. Appendix 2 - Definitions

Table 5.1. Abbreviations

Abbreviation	Definition
NA	Not Available